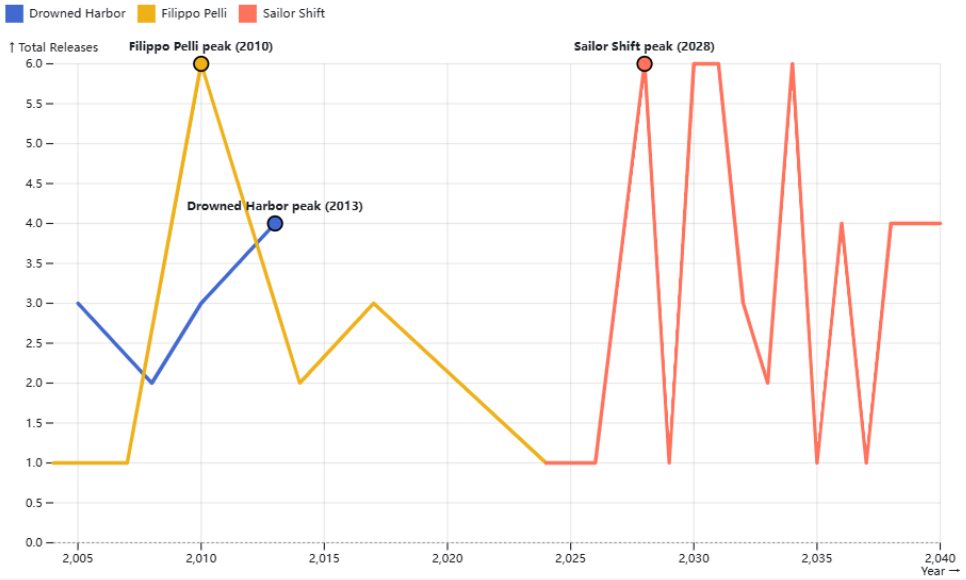
Task 3: Oceanus Folk Artist Analysis

```
timeline = ▸ Array(26) [Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Ob
```

Select: ◉ total_releases ○ notable_releases

Highlight artist: [All ▾]




```
{//Takes the raw timeline dataset and converts fields into numbers
  const cleanTimeline = timeline
    .map((d) => ({
      name: d.name,
      total_releases: +d.total_releases,
      notable_releases: +d.notable_releases,
      release_date: +d.release_date}))
    .filter((d) => !isNaN(d.release_date));//removes rows where release_date is invalid

  //Finding each artist's peak productivity
  const peaks = Array.from(d3.group(cleanTimeline, (d) => d.name),
    ([name, values]) => values.reduce((a, b) => (b.total_releases > a.total_releases ? b : a)));
  //Finding each artist's breakthrough
  const breakthroughs = Array.from(d3.group(cleanTimeline, (d) => d.name),
    ([name, values]) => values.reduce((a, b) => (b.notable_releases > a.notable_releases ? b : a)));

  return Plot.plot({
    title: "Career Growth vs Success",
    subtitle: "Interactive exploration of artist productivity and impact",
    width: 800,
    height: 450,
    x: {label: "Year",grid: true},
    y: {label:selection === "total_releases" ? "Total Releases" : "Notable Releases",grid: true},
    color: {legend: true,label: "Artist"},
    marks: [
      // baseline, a horizontal line at y = 0
      Plot.ruleY([0], { stroke: "#999", strokeDasharray: "2,2" }),
      // line
      Plot.line(cleanTimeline, {
        x: "release_date", y: selection,
        stroke: "name",
        curve: "linear",
        strokeWidth: (d) => (focus === "All" || d.name === focus ? 3 : 1.5), //Dynamic thickness
        opacity: (d) => (focus === "All" || d.name === focus ? 1 : 0.2) //Dynamic opacity
      }),
```

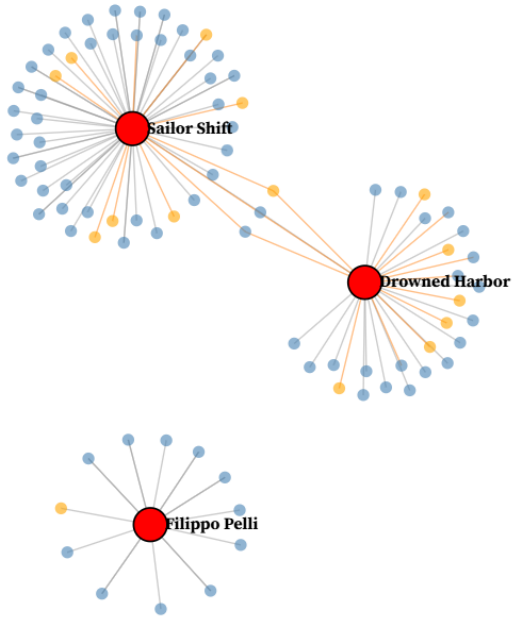
```
      // ONLY peaks for total_releases
      ...(selection === "total_releases"
        ? [Plot.dot(peaks, {
              x: "release_date", y: "total_releases",
              fill: "name",
              r: 6,
              stroke: "black",
              strokeWidth: 1.5}),
            Plot.text(peaks, {
              x: "release_date", y: "total_releases",
              text: (d) => `${d.name} peak (${d.release_date})`,
              dy: -14,
              fontWeight: "bold"})]: []),

      // ONLY breakthroughs for notable_releases
      ...(selection === "notable_releases"
        ? [Plot.dot(breakthroughs, {
              x: "release_date", y: "notable_releases",
              fill: "name",
              r: 6,
              stroke: "black",
              strokeWidth: 1.5}),
            Plot.text(breakthroughs, {
              x: "release_date", y: "notable_releases",
              text: (d) => `${d.name} breakthrough (${d.release_date})`,
              dy: -14,
              fill: "green",
              fontWeight: "bold"})]: []),

      // tooltip
      Plot.tip(
        cleanTimeline,
        Plot.pointer({
          x: "release_date",y: selection,
          title: (d) => `${d.name}
            Year: ${d.release_date}
            Total: ${d.total_releases}
            Notable: ${d.notable_releases}`
        }))
    ]
  });
}
```

```
network = ▸ Object {nodes: Array(90), edges: Array(107)}
```

```
network = FileAttachment("network@1.json").json()
```



```
{
  const mainArtistNames = new Set([
    "Sailor Shift",
    "Filippo Pelli",
    "Drowned Harbor"
  ]);
  //Cleaning node data
  const nodes = network.nodes.map((d) => ({
    id: d.id,
    name: d.name || "",
    type: d.node_type
  }));
  //Cleaning edge data
  const links = network.edges.map((d) => ({
    source: d.source,
    target: d.target,
    type: d.edge_type
  }));

  // Get IDs of main artists
  const mainIds = new Set(
    nodes.filter((d) => mainArtistNames.has(d.name)).map((d) => d.id)
  );

  // Keep only links connected to main artists
  const linksFiltered = links.filter(
    (d) => mainIds.has(d.source) || mainIds.has(d.target)
  );

  // Keep only nodes that appear in those links
  const nodeIds = new Set(linksFiltered.flatMap((d) => [d.source, d.target]));
  const filteredNodes = nodes.filter((d) => nodeIds.has(d.id)); //keeps only relevant nodes
  const width = 1000;
  const height = 700;
  const svg = d3.create("svg").attr("width", width).attr("height", height);
  const simulation = d3
    .forceSimulation(filteredNodes)
    .force("link",d3.forceLink(linksFiltered).id((d) => d.id).distance(70)) //Connected nodes stay 70px apart
    .force("charge", d3.forceManyBody().strength(-80)) // less explosion, Prevents overlap
    .force("center", d3.forceCenter(width / 2, height / 2)) //Keeps graph centered.
    .force("x", d3.forceX(width / 2).strength(0.05)) // pull to center
    .force("y", d3.forceY(height / 2).strength(0.05)) // pull to center
    .force("collision", d3.forceCollide().radius(10));//Prevents circles from overlapping

  const link = svg //Drawing links
    .append("g")
    .selectAll("line")
    .data(linksFiltered)
    .join("line")
    .attr("stroke", (d) => {//Color
      if ([
          "PerformerOf",
          "ComposerOf",
          "ProducerOf",
          "LyricistOf",
          "MemberOf"].includes(d.type))
      {return "#999";} //grey// collaboration
      return "#ff7f0e";}) //orange// influence
    .attr("stroke-opacity", 0.5)
    .attr("stroke-width", 1.2);

  const node = svg //Drawing nodes
    .append("g")
    .selectAll("circle")
    .data(filteredNodes)
    .join("circle")
    .attr("r", (d) => (mainArtistNames.has(d.name) ? 14 : 5)) //Main artists are much larger.
    .attr("fill", (d) => {
      if (mainArtistNames.has(d.name)) return "red"; // MAIN
      if (d.type === "MusicalGroup") return "orange";
      return "steelblue";
    })
    .attr("opacity", (d) => (mainArtistNames.has(d.name) ? 1 : 0.6)) //Main artists are fully visible
    .attr("stroke", (d) => (mainArtistNames.has(d.name) ? "black" : "none"))
    .attr("stroke-width", 1.5);

  node.on("mouseover", (event, d) => {//Highlight connected links and nodes
    link.attr("stroke-opacity", (l) => l.source.id === d.id || l.target.id === d.id ? 1 : 0.05);
    node.attr("opacity", (n) => n.id === d.id || linksFiltered.some((l) =>
          (l.source.id === d.id && l.target.id === n.id) ||
          (l.target.id === d.id && l.source.id === n.id))? 1: 0.2);});
```

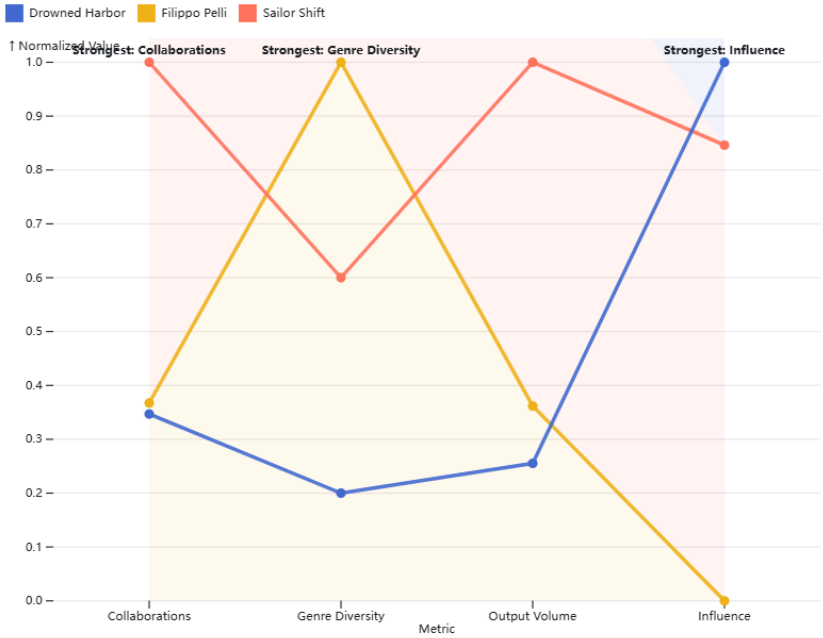
```
  node.on("mouseout", () => { //Mouse out reset
    link.attr("stroke-opacity", 0.5);
    node.attr("opacity", 0.6);});

  const label = svg
    .append("g")
    .selectAll("text")
    .data(filteredNodes.filter((d) => mainArtistNames.has(d.name))) //Only main artists get labels
    .join("text")
    .text((d) => d.name)
    .attr("font-size", 14)
    .attr("font-weight", "bold")
    .attr("fill", "black")
    .attr("dx", 12)
    .attr("dy", 4);
  simulation.on("tick", () => {
    filteredNodes.forEach((d) => { //Keeping nodes inside bounds
      d.x = Math.max(20, Math.min(width - 20, d.x));
      d.y = Math.max(20, Math.min(height - 20, d.y));
    });
    link //Updating line positions
      .attr("x1", (d) => d.source.x)
      .attr("y1", (d) => d.source.y)
      .attr("x2", (d) => d.target.x)
      .attr("y2", (d) => d.target.y);
    node.attr("cx", (d) => d.x).attr("cy", (d) => d.y); //Updating node positions
    label.attr("x", (d) => d.x).attr("y", (d) => d.y); //Updating labels
  });
  return svg.node();
}
```

```
metrics = ▸ Array(3) [Object, Object, Object, columns: Array(5)]
```

```
metrics = FileAttachment("metrics@1.csv").csv({ typed: true })
```

Artist Profile Comparison: Strengths Across Key Metrics




```
{
  const keys = ["collaborations","influence_count","genre_diversity","output_volume"];
  // reorder for better visual flow
  const orderedKeys = ["collaborations","genre_diversity","output_volume","influence_count"];
  const labelMap = {collaborations: "Collaborations",genre_diversity: "Genre Diversity",
    output_volume: "Output Volume",influence_count: "Influence"};

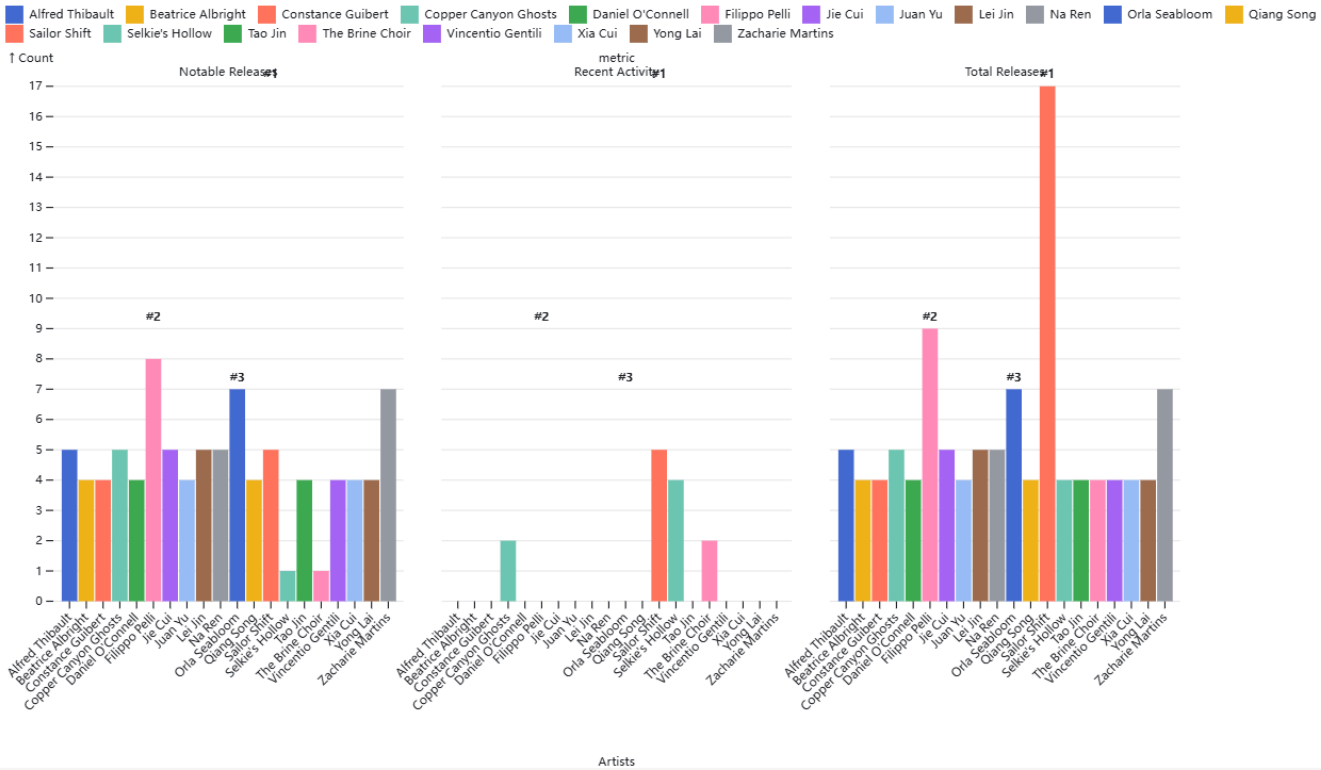
  // normalize
  const maxValues = {}; orderedKeys.forEach((k) => {maxValues[k] = d3.max(metrics, (d) => d[k]);});
  const normalized = metrics.flatMap((d) =>
    orderedKeys.map((k) => ({name: d.name, metric: k, value: d[k] / maxValues[k]})));
  const peaks = Array.from(d3.group(normalized, (d) => d.name),
    ([name, values]) => values.reduce((a, b) => (b.value > a.value ? b : a)));
```

```
  return Plot.plot({
    title: "Artist Profile Comparison: Strengths Across Key Metrics",
    width: 700,
    height: 500,
    x: {label: "Metric", domain: orderedKeys, tickFormat: (d) => labelMap[d]},
    y: {label: "Normalized Value", domain: [0, 1], grid: true},
    color: { legend: true },
    marks: [
      Plot.areaY(normalized, {
        x: "metric",
        y: "value",
        fill: "name",
        z: "name",
        fillOpacity: 0.08,
        curve: "linear"}),
      Plot.line(normalized, {
        x: "metric",
        y: "value",
        stroke: "name",
        strokeWidth: 3,
        strokeOpacity: 0.9}),
      Plot.dot(normalized, {
        x: "metric",
        y: "value",
        fill: "name",
        r: 4}),
      Plot.text(peaks, {
        x: "metric",
        y: "value",
        text: (d) => `Strongest: ${labelMap[d.metric]}`,
        dy: -10,
        fontWeight: "bold"})]
  });
}
```

```
potentialStars = ▸ Array(30) [Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Obje
```

```
potentialStars = await FileAttachment("potential_stars_all@1.csv").csv({typed: true})
```

Oceanus Folk Artists: Key Metrics for Rising Star Potential



```
{
  const top20 = potentialStars
    .sort((a, b) => b.score - a.score) // rank by score
    .slice(0, 20); //show top 20
  const long = top20.flatMap((d) => [
    { name: d.name, metric: "Total Releases", value: d.total_releases },
    { name: d.name, metric: "Notable Releases", value: d.notable_releases },
    { name: d.name, metric: "Recent Activity", value: d.recent_activity }
  ]);

  return Plot.plot({
    title: "Oceanus Folk Artists: Key Metrics for Rising Star Potential",
    width: 1000, height: 600,
    x: { label: "Artists", tickRotate: -45 },
    y: { label: "Count", grid: true },
    color: { legend: true },
    marks: [
      Plot.barY(long, {
        x: "name",
        y: "value",
        fill: "name",
        fx: "metric",
        title: (d) => `${d.metric}: ${d.value}`}),
      Plot.text(long
          .filter((d) => d.metric === "Total Releases")
          .sort((a, b) => b.value - a.value)
          .slice(0, 3),
        {x: "name", y: "value", text: (d, i) => `#${i + 1}`, dy: -10, fontWeight: "bold"})],
    marginBottom: 140});
}
```

```
top_stars = ▸ Array(30) [Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, C
```

```
data = ▸ Array(30) [Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object, Object
```

```
predictedStars = ▸ Array(3) [Object, Object, Object]
```

Identifying Rising Stars in Oceanus Folk

Quadrant classification with weighted rising star score




```
{
  const data = top_stars.map((d) => ({
    name: d.name,
    total_releases: +d.total_releases,
    notable_releases: +d.notable_releases,
    recent_activity: +d.recent_activity,
    score: +d.score}));

  // Normalize score for color scale
  const maxScore = d3.max(data, (d) => d.score); data.forEach((d) => (d.normScore = d.score / maxScore));
  const predictedStars = data.sort((a, b) => b.score - a.score).slice(0, 3);
  const xMid = d3.median(data, (d) => d.total_releases) + 2;
  const yMid = d3.median(data, (d) => d.notable_releases);
  const classify = (d) => {
    if (d.total_releases < xMid && d.notable_releases > yMid) return "rising";
    if (d.total_releases >= xMid && d.notable_releases > yMid) return "superstar";
    if (d.total_releases >= xMid && d.notable_releases <= yMid) return "established";
    return "emerging"; // fixed naming
  };
```

```
  return Plot.plot({
    title: "Identifying Rising Stars in Oceanus Folk",
    subtitle: "Quadrant classification with weighted rising star score",
    width: 800,
    height: 500,
    x: {label: "Total Releases", grid: true, domain: [0, d3.max(data, (d) => d.total_releases)]},
    y: {label: "Notable Releases", grid: true, domain: [0, d3.max(data, (d) => d.notable_releases)]},
    color: {legend: true,label: "Rising Star Score"},// Color represents score
    symbol: {
      legend: true,
      domain: ["rising", "superstar", "established", "emerging"],
      range: ["triangle", "star", "square", "circle"]},// Shape represents category
    marks: [
      Plot.ruleX([xMid], { stroke: "black", strokeDasharray: "4,4" }),// Quadrant lines
      Plot.ruleY([yMid], { stroke: "black", strokeDasharray: "4,4" }),
      Plot.dot(data, { // All artists
        x: (d) => d.total_releases + (Math.random() - 0.5) * 0.3,
        y: (d) => d.notable_releases + (Math.random() - 0.5) * 0.3,
        r: (d) => d.recent_activity * 2 + 3,
        fill: "normScore", // color = score
        symbol: (d) => classify(d), // shape = category
        opacity: 0.8,
        title: (d) => `${d.name}
        Releases: ${d.total_releases}
        Notable: ${d.notable_releases}
        Recent: ${d.recent_activity}
        Score: ${d.score.toFixed(1)}`}),
      Plot.text(data.sort((a, b) => b.score - a.score).slice(0, 3), {// Top 3 labels
        x: "total_releases",
        y: "notable_releases",
        text: (d, i) => `#${i + 1}`,
        dy: -18,
        fontWeight: "bold",
        fill: "black"}),
      Plot.text([// Quadrant labels
          { x: xMid * 0.5, y: yMid * 1.5, label: "Rising Stars" },
          { x: xMid * 1.5, y: yMid * 1.5, label: "Superstars" },
          { x: xMid * 1.5, y: yMid * 0.5, label: "Established" },
          { x: xMid * 0.5, y: yMid * 0.5, label: "Emerging" }],
        {x: "x", y: "y", text: "label", fontWeight: "bold", fill: "gray"}),
      Plot.text(data.sort((a, b) => b.score - a.score).slice(0, 5), {  // Top 5 names
        x: "total_releases", y: "notable_releases", text: (d) => d.name, dy: -10, fontSize: 10})]
  });
}
```

Predicted Rising Stars

1. Copper Canyon Ghosts
2. Selkie's Hollow
3. The Brine Choir

Task 3: Analysis of Rising Stars in Oceanus Folk Music

Data Analysis

Before being imported into Observable for visualisation, I first cleaned and processed the data in R using the **jsonlite** and **dplyr** packages, and produced several important csv files. To make the data easier to work with, I changed all column names to lowercase and replaced spaces with underscores to avoid issues when joining node and edge tables. I also changed the **release_date** field from text into numeric format using **as.numeric()** so I could analyse trends over time more easily. Whereas records with missing or invalid release dates were only removed when the analysis involved time. This was done using **(filter(!is.na(release_date)))** so that other parts of the analysis could still keep as much data as possible.

To keep the investigation focused, the dataset was filtered to artists associated with the Oceanus Folk genre. Songs labelled as "Oceanus Folk" were linked back to performers and composers through relationships such as PerformerOf and ComposerOf. Several artist-level variables were then created from the joined node-edge structure, including:

- **"total_releases"**
- **"notable_releases"**
- **"career_length"**
- **"recent_activity (post-2035 output)""**

These measures were used to represent artist productivity and overall influence within the Oceanus Folk. Whereas, **d3.group()** and **reduce()** were used to identify breakthrough periods and peak release years for each artist. So to identify potential rising stars, a weighted scoring model was developed:

score = 3 × recent_activity + 1.5 × notable_releases + 0.5 × total_releases

The scoring I have applied emphasizes more on recent momentum, helping prevent long-established artists from dominating the rankings based only on older releases without any recent activities.

For the network analysis, relationships connected to collaboration and influence, including **PerformerOf, MemberOf, InStyleOf, and CoverOf** were combined into a simplified network graph. This is because the full network at large scale made the visualisation difficult to read, and thus, ego-network filtering was used. Additional measures such as **collaborations, influence_count, genre_diversity, and output_volume** were then calculated from the connected relationships between artists.

Visual Design

The visualisations were designed based on ideas from Cleveland and McGill (1984) and Munzner (2014), to make the graphs easier to read, compare, and explore with the large data given for my task.

Graph 1: Timeline - Career Growth vs Success

A timeline chart was used to compare the total releases and notable releases, where the overall trends present via lines, with highlights on breakthrough year and peak years. Interactive controls were added to allow users to switch between productivity and impact views, for easy tracking of different artists developed over time. For example, Sailor Shift shows a clear increase in activity after 2028, reflecting the effect of her first breakthrough.

Graph 2: Network Graph - Collaboration and Influence

A force-directed graph was created to explore collaboration and influence relationships. The position of each node reflects how closely artists are connected, while different edge colours separate collaboration links from influence links. And in my case, Sailor Shift, Filippo Pelli, and Drowned Harbor were focused. Hover interactions were added to visual nearby connections more clearly. From the graph, Sailor Shift appears as the main hub with most connections, while Drowned Harbor appears smaller but shows increasing connectivity. Interestingly, Filippo Pelli is not directly connected to Sailor Shift in the filtered network graph.

Graph 3: Multi-Metric Profile Comparison

A normalised multi-line chart was created to allow fair comparison between metrics with different scales across collaborations, genre diversity, output volume, and influence count. The chart highlights profile differences: Sailor Shift performs strongly in influence metrics, while Filippo Pelli shows higher genre diversity.

Graph 4: Rising Star Metrics

A faceted bar chart was used to compare artists across total releases, notable releases, and recent activity. Separating panels reduces clutter and improves readability. The chart suggests that recent activity is an important indicator of future success, since several artists with shorter careers still rank highly due to strong recent output.

Graph 5: Quadrant Classification Model

A scatter plot maps total releases against notable releases, with point size representing recent activity. Dashed reference lines divide the plot into four categories: rising, superstar, established, and emerging artists. This makes the different artist career patterns easier to compare; Sailor Shift appears in the superstar category; Filippo Pelli shows balanced high performance; and Copper Canyon Ghosts falls within the rising stars region.

Result

Several patterns observed from the visualisations reflect the criteria of a rising star in the Oceanus Folk industry. From the timeline visualisation, we can see that major increases in popularity often happen after breakthrough releases. Sailor Shift shows the clearest example of this, with both total releases and notable releases increasing rapidly after 2028. From the network graph, it can be seen that influence is concentrated around a small number of highly connected artists. Sailor Shift appears as the main part in the network, while Drowned Harbor shows growing connectivity and Filippo Pelli does not appear connected to Sailor Shift.

The multi-metric comparison reminds us that success does not come from a single factor. Artists who stand out need to have balanced strengths across different metrics, like Sailor Shift. In addition, the quadrant classification chart also suggests that rising stars are more strongly linked to recent activity than to long careers with large output. For example, Sailor Shift and Filippo Pelli appear closer to superstar status because much of their success comes from earlier career growth but not recent.

From the analysis results, Copper Canyon Ghosts, Selkie's Hollow, and The Brine Choir were most likely to be the future Oceanus Folk stars over the next five years, with their strong recent activities and increasing visibility in the rising star metrics panel. In conclusion, recent momentum and artist connections are more important than a historically long career.

Reflection

The main challenge I faced was creating the network data. At first, I found that the full network graph had too many nodes and edges overlapped, which is unreadable. To solve this, I used ego-network filtering by focusing only on the 3 comparing artists and their connections, but it also meant that some wider relationships in the network were lost.

Another difficulty was measuring artist success fairly. At first, I looked at the total releases and found that they tended to favour artists with much longer careers, so I included recent activity and notable releases to form a scoring system to better reflect the future rising stars prediction that have momentum and influence nowadays instead of already strong historical data.

In addition, I also dealt with metrics that were on very different scales. I used normalisation to bring the values into a similar range, which made comparisons between artists easier. However, the actual differences in scale might be lost after the normalization.

Several more designs were tested but were rejected by me during finalizing ; radar charts for profile comparison which are difficult to interpret and full networks which are too cluttered. So in the end, I selected the final visual designs for reporting my analysis based on clarity and meaningful details.

If I had more time, I would like to study more advanced network analysis methods to explore extra influence spreads between artists. I would also like to explore predictive modelling approaches to improve my current weighted scoring method used for identifying future rising stars.

References

- Cleveland, W. S., & McGill, R. (1984). Graphical perception.
- Munzner, T. (2014). Visualization Analysis and Design.


```
# Task 3: Analysis of Rising Stars in Oceanus Folk Music
## Data Analysis
Before being imported into Observable for visualisation, I first cleaned and processed the data in R using the **jsonlite** and **dplyr**
packages, and produced several important csv files. To make the data easier to work with, I changed all column names to lowercase and
replaced spaces with underscores to avoid issues when joining node and edge tables. I also changed the **release_date** field from text
```